

Principales Códigos de Error y Estado en un proyecto web

- **200 OK:** La solicitud ha tenido éxito
- **301/302 Redirecciones:** Indican que el recurso se ha movido temporal o permanentemente
- **400 Bad Request:** El servidor no entiende la petición por sintaxis incorrecta
- **401 Unauthorized:** Requiere autenticación para acceder
- **403 Forbidden:** El servidor entiende la petición pero se niega a autorizarla
- **404 Not Found:** El servidor no encuentra el recurso solicitado (enlace roto)
- **500 Internal Server Error:** Error genérico del servidor
- **502 Bad Gateway:** El servidor recibió una respuesta inválida de otro servidor
- **503 Service Unavailable:** El servidor no está disponible actualmente (mantenimiento o sobrecarga) .
- **504 Gateway Timeout:** El servidor no recibió una respuesta a tiempo de otro servidor

1. Creación y Configuración del Proyecto Angular

Necesitas tener Node.js y el CLI de Angular instalados.

Paso 1: Instalar el CLI de Angular

Si aún no lo tienes, instala el *Command Line Interface* (CLI) de Angular globalmente:

desde consola CMD:

```
npm install -g @angular/cli
```

```
npm install @ngneat/hot-toast /// Esto agrega el manejo de mensajes flotantes mas modernos al usuario.
```

```
npm install @ngneat/overview --legacy-peer-deps
```

```
npm install @angular/animations --legacy-peer-deps
```

Paso 2: Crear el Nuevo Proyecto

Abre tu terminal en el directorio donde deseas crear la carpeta del *frontend* y usa el comando

```
ng new superhero-frontend --strict --routing
```

ó si se quiere establecer de una vez que quiere usar SCSS y routing ejecutar lo siguiente:

```
ng new superhero-frontend --strict --routing=true --style=scss
```

El CLI te hará algunas preguntas:

Pregunta	Respuesta Sugerida
Would you like to add Angular routing?	Yes (Aceptado en el comando --routing)
Which stylesheet format would you like to use?	SCSS (Recomendado) o CSS

Esto creará la carpeta *superhero-frontend* y configurará un proyecto Angular básico con TypeScript.

Paso 3: Iniciar la Aplicación

Navega a la nueva carpeta e inicia el servidor de desarrollo de Angular:

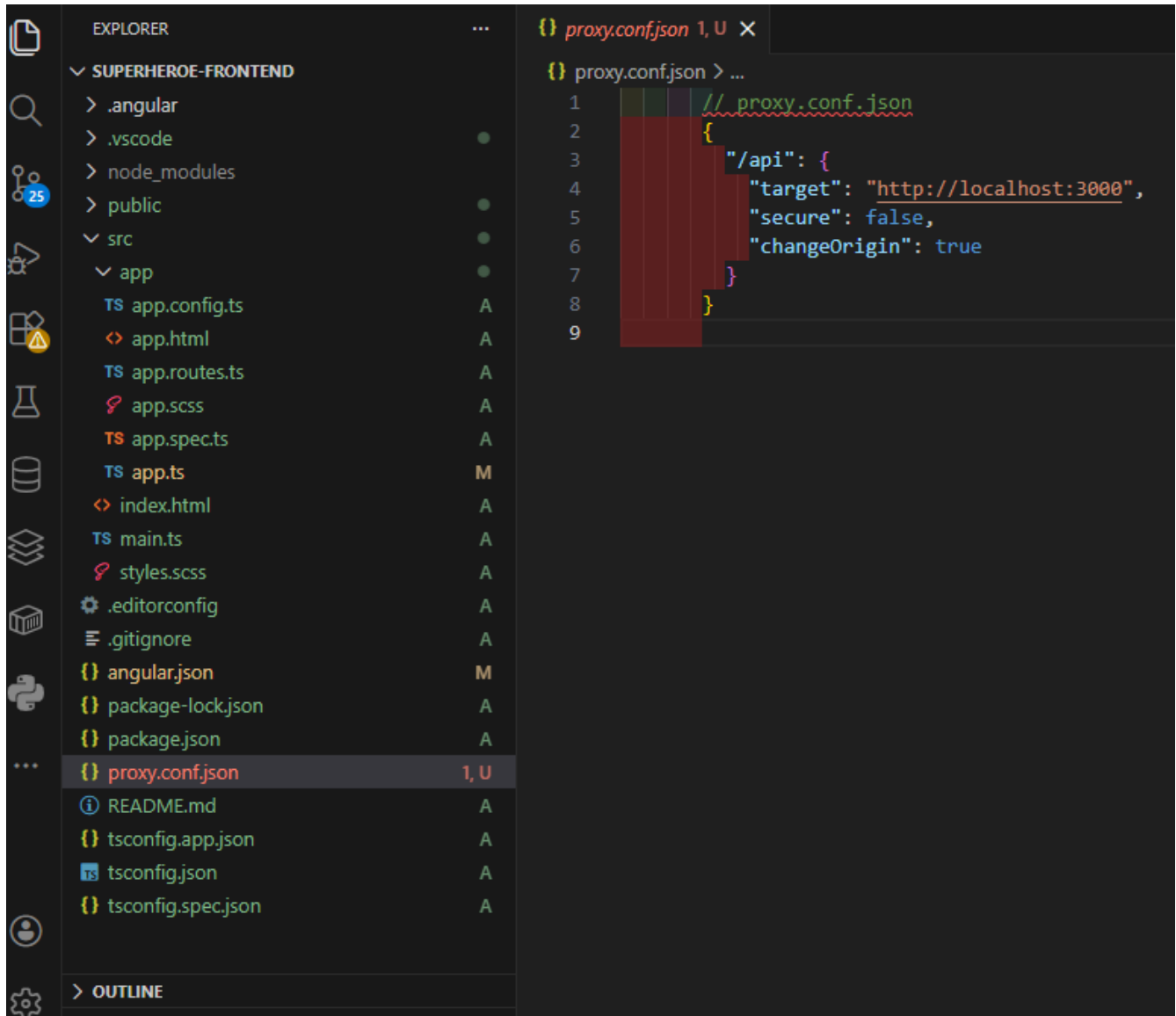
CMD:

```
cd superhero-frontend
```

```
ng serve
```

Tu *frontend* estará corriendo, típicamente en `http://localhost:4200`

Así se debe de ver la ramificación en tu visual code.



```
{  
  "proxy": {  
    "target": "http://localhost:3000",  
    "secure": false,  
    "changeOrigin": true  
  }  
}
```

2. Integración y Conexión con el Backend

El reto principal es manejar la comunicación segura entre el *frontend* (puerto 4200) y el *backend* (puerto 3000) y configurar el manejo de peticiones HTTP.

A. Configurar el Proxy de Angular (Crucial)

Para evitar errores de CORS en desarrollo, configuraremos Angular para que redirija las solicitudes API a tu *backend*.

1. **Crea el archivo de configuración del proxy:** En la raíz de tu proyecto Angular (superheroefrontend/), crea un archivo llamado **proxy.conf.json**:

JSON

```
// proxy.conf.json
{
  "/api": {
    "target": "http://localhost:3000",
    "secure": false,
    "changeOrigin": true
  }
}
```

Asegúrate de que `http://localhost:3000` se a el puerto donde corre tu servidor Express. (Tu back-end)

NOTA: estando en modo CMD desde carpeta raíz del proyecto: superheroefrontend , se puede ejecutar el comando para abrir el visual code desde esta carpeta usando la frase “code” espacio y un punto “.” de la siguiente manera: “code .” y Enter , esto abrirá el editor. De esta manera crear los documentos necesarios.

Configura Angular para usar el proxy: Abre el archivo **angular.json** en la raíz del proyecto. Busca la sección **serve** dentro de projects -> superheroefrontend -> architect -> serve -> options y añade la propiedad proxyConfig:

JSON

```
// angular.json (Fragmento)
"serve": {
  "builder": "@angular-devkit/build-angular:dev-server",
  "options": {

    "proxyConfig": "proxy.conf.json" // <--- AGREGA ESTA LÍNEA
  },
  // ...
},
```

Reinicia el servidor: Detén (Ctrl+C) y vuelve a iniciar el servidor de desarrollo para aplicar los cambios:

Bash(cmd)

ng serve , si no funciona este comando usar este otro: ng start

Ahora, cualquier solicitud HTTP que comience con /api será redirigida a tu *backend* en el puerto 3000.

I. Frontend del Catálogo (HTML/TypeScript)

Este frontend se centrará en consumir los endpoints que creamos (/api/heroes/catalog y /api/heroes/favorites). Usaremos un diseño responsivo básico con CSS Grid para lograr la distribución de 4 columnas solicitada.

A. Estructura HTML (index.html) app.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Catálogo de Superhéroes - Backend NodeJS</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Catálogo de Superhéroes ⚡</h1>
    <nav>
      <button id="show-catalog-btn">Ver Catálogo General (Grid)</button>
      <button id="show-favorites-btn">Mis Héroes Favoritos</button>
    </nav>
  </header>

  <main>
    <h2>Superhéroes Disponibles</h2>

    <div id="hero-grid" class="hero-grid">

      <p id="message" style="text-align: center; margin-top: 20px; font-weight: bold;"></p>

    </div>

    <script src="dist/app.js"></script>
  </body>
</html>
```

B. Estilos CSS (styles.css)

```
body {
```

```
font-family: sans-serif;
margin: 0;
padding: 0;
background-color: #f4f4f9;
}
```

```
header {
  background-color: #3f51b5;
  color: white;
  padding: 20px;
  text-align: center;
}
```

```
nav button {
  background-color: #ff9800;
  color: white;
  border: none;
  padding: 10px 20px;
  margin: 5px;
  cursor: pointer;
  border-radius: 4px;
  transition: background-color 0.3s;
}
```

```
nav button:hover {
  background-color: #f57c00;
}
```

```
main {
  padding: 20px;
  max-width: 1200px;
  margin: 0 auto;
}
```

```
/* --- GRID Responsivo de 4 Columnas --- */
```

```
.hero-grid {
  display: grid;
  /* Por defecto, una columna */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
  margin-top: 20px;
}
```

```
.hero-card {
  background-color: white;
  border: 1px solid #ddd;
  border-radius: 8px;
  overflow: hidden;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
}
```

```

    transition: transform 0.2s;
  }

.hero-card:hover {
  transform: translateY(-5px);
}

.hero-card img {
  width: 100%;
  height: 200px;
  object-fit: cover;
}

.hero-content {
  padding: 15px;
}

.hero-content h3 {
  margin-top: 0;
  color: #3f51b5;
}

.hero-content p {
  font-size: 0.9em;
  margin: 5px 0;
}

/* Media query para asegurar 4 columnas en pantallas grandes */
@media (min-width: 1000px) {
  .hero-grid {
    /* Fija las columnas a 4 */
    grid-template-columns: repeat(4, 1fr);
  }
}

```

C. Lógica TypeScript (app.ts)

Asume que ya tienes configurado TypeScript y que tu archivo se compilará a dist/app.js

// app.ts (Debe ser compilado a JavaScript antes de ser usado en index.html)

```

const API_BASE_URL = 'http://localhost:3000/api'; // Ajusta si tu puerto es diferente
const GRID_CONTAINER = document.getElementById('hero-grid') as HTMLElement;
const MESSAGE_ELEMENT = document.getElementById('message') as HTMLElement;
const AUTH_TOKEN = 'SIMULATED_JWT_TOKEN'; // *Simulación*: Aquí se guardaría el token al
hacer login.

```

// 1. Interfaz de Datos (Basada en tu esquema Prisma)

```
interface Superheroe {
  id: number;
  nombre: string;
  poder: string;
  fortaleza: string;
  resistencia: string;
  debilidad: string;
  imagen_url: string;
}
```

// 2. Función para renderizar un solo héroe

```
function createHeroCard(hero: Superheroe): HTMLDivElement {
  const card = document.createElement('div');
  card.className = 'hero-card';
  card.innerHTML = `
    
    <div class="hero-content">
      <h3>${hero.nombre}</h3>
      <p><strong>Poder:</strong> ${hero.poder.substring(0, 50)}...</p>
      <p><strong>Fortaleza:</strong> ${hero.fortaleza.substring(0, 50)}...</p>
      <p><strong>Resistencia:</strong> ${hero.resistencia.substring(0, 50)}...</p>
      <p><strong>Debilidad:</strong> ${hero.debilidad.substring(0, 50)}...</p>
      <button onclick="addToFavorites(${hero.id})">Añadir a Favoritos ★</button>
    </div>
  `;
  return card;
}
```

// 3. Función para mostrar el listado completo (Catálogo)

```
async function fetchCatalog() {
  GRID_CONTAINER.innerHTML = "";
  MESSAGE_ELEMENT.textContent = 'Cargando catálogo...';
  try {
    const response = await fetch(`${API_BASE_URL}/heroes/catalog`);
    if (!response.ok) throw new Error('Error al obtener el catálogo.');


const heroes: Superheroe[] = await response.json();



if (heroes.length === 0) {
    MESSAGE_ELEMENT.textContent = 'No hay superhéroes en el catálogo.';
    return;
  }



heroes.forEach(hero => {
    GRID_CONTAINER.appendChild(createHeroCard(hero));
  });



MESSAGE_ELEMENT.textContent = `Catálogo cargado. Mostrando ${heroes.length} héroes.`;



} catch (error) {
  console.error("Fetch error:", error);
}


```



```

    MESSAGE_ELEMENT.textContent = 'Hubo un error al conectar con el servidor.';
  }
}

// 4. Función para mostrar los favoritos del usuario (Requiere token)
async function fetchFavorites() {
  GRID_CONTAINER.innerHTML = "";
  MESSAGE_ELEMENT.textContent = 'Cargando tus favoritos...';

  if (!AUTH_TOKEN) {
    MESSAGE_ELEMENT.textContent = 'Debes iniciar sesión para ver tus favoritos.';
    return;
  }

  try {
    const response = await fetch(`${API_BASE_URL}/heroes/favorites`, {
      headers: {
        'Authorization': AUTH_TOKEN // Usando el token simulado
      }
    });

    if (response.status === 401) throw new Error('Token inválido. Inicia sesión.');
    if (!response.ok) throw new Error('Error al obtener favoritos.');

    const heroes: Superhero[] = await response.json();

    if (heroes.length === 0) {
      MESSAGE_ELEMENT.textContent = 'Aún no tienes héroes favoritos. ¡Explora el catálogo!';
      return;
    }

    heroes.forEach(hero => {
      GRID_CONTAINER.appendChild(createHeroCard(hero));
    });
    MESSAGE_ELEMENT.textContent = `Mostrando ${heroes.length} héroes favoritos.`;
  } catch (error: any) {
    console.error("Fetch error:", error);
    MESSAGE_ELEMENT.textContent = error.message;
  }
}

// 5. Función para añadir a favoritos (simulación)
(window as any).addToFavorites = async function(heroId: number) {
  if (!AUTH_TOKEN) {
    alert("Debes iniciar sesión para añadir favoritos.");
    return;
  }

  try {

```

```

const response = await fetch(`${API_BASE_URL}/heroes/favorites`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': AUTH_TOKEN
  },
  body: JSON.stringify({ heroId })
});

const result = await response.json();
if (response.ok) {
  alert(result.message); // Héroe agregado con éxito
} else {
  alert("Error: " + (result.error || result.message));
}
} catch (error) {
  alert("Error de conexión al añadir favorito.");
}
};

// 6. Event Listeners y Carga Inicial
document.addEventListener('DOMContentLoaded', () => {
  document.getElementById('show-catalog-btn')?.addEventListener('click', fetchCatalog);
  document.getElementById('show-favorites-btn')?.addEventListener('click', fetchFavorites);

  // Carga inicial al abrir la página
  fetchCatalog();
});

```

Script Seeder (Prisma)

Este script llenará tu tabla catsuperheroe con datos de ejemplo para que puedas probar el frontend y el grid de 4x3 inmediatamente.

A. Configuración

1. Crea la carpeta db/seeds si no existe.
2. Crea el archivo db/seeds/01_superheroes.ts.
3. Asegúrate de tener instalado ts-node y typescript como dev dependencies (npm install -D ts-node typescript).

B. Código del Seeder (db/seeds/01_superheroes.ts)

// seeds/01_superheroes.ts (o la ruta que uses para tus seeds de Knex)

```

import { Knex } from 'knex';

// -----
// 1. Datos de Semilla (La lista de Superhéroes)
// -----

const superHeroesData = [
  {
    nombre: 'Superman',
    poder: 'Vuelo, Súper Fuerza, Visión de calor',
    fortaleza: 'Sol Amarillo',
    resistencia: 'Extremadamente alta',
    debilidad: 'Kryptonita',
    imagen_url: 'http://example.com/superman.jpg'
  },
  {
    nombre: 'Wonder Woman',
    poder: 'Fuerza sobrehumana, Vuelo, Lazo de la Verdad',
    fortaleza: 'Su voluntad y su entrenamiento',
    resistencia: 'Muy alta',
    debilidad: 'Armas punzantes (anteriormente)',
    imagen_url: 'http://example.com/wonderwoman.jpg'
  },
  {
    nombre: 'Batman',
    poder: 'Intelecto de genio, Maestría en combate',
    fortaleza: 'Su preparación y gadgets',
    resistencia: 'Humana máxima',
    debilidad: 'Mortalidad y traumas',
    imagen_url: 'http://example.com/batman.jpg'
  },
  {
    nombre: 'Flash (Barry Allen)',
    poder: 'Súper Velocidad, Capacidad de vibrar a través de la materia',
    fortaleza: 'La Speed Force',
    resistencia: 'Acelerada',
    debilidad: 'Frío extremo',
    imagen_url: 'http://example.com/flash.jpg'
  },
  {
    nombre: 'Spider-Man (Peter Parker)',
    poder: 'Fuerza y agilidad proporcionales a una araña, Sentido arácnido',
    fortaleza: 'Su sentido arácnido y lanzaredes',
    resistencia: 'Alta',
    debilidad: 'Ciertos sonidos de alta frecuencia',
    imagen_url: 'http://example.com/spiderman.jpg'
  },
  {
    nombre: 'Iron Man (Tony Stark)',

```

```

    poder: 'Genio multimillonario, Armadura avanzada',
    fortaleza: 'Su inteligencia y recursos',
    resistencia: 'La de su armadura',
    debilidad: 'Su ego y dependencia de la armadura',
    imagen_url: 'http://example.com/ironman.jpg'
  },
  // Puedes añadir más héroes aquí...
];

// -----
// 2. Funciones de Knex para Seed
// -----

/**
 * Función que inserta los datos. Es requerida por Knex.
 * @param knex Instancia de Knex
 */
export async function seed(knex: Knex): Promise<void> {
  const TABLE_NAME = 'catsuperheroe';

  // Borra *TODOS* los registros existentes de la tabla antes de insertar
  await knex(TABLE_NAME).del();

  // Inserta los nuevos datos
  await knex(TABLE_NAME).insert(superHeroesData);

  console.log('✅ ${superHeroesData.length} Superhéroes insertados en la tabla
'${TABLE_NAME}'.');
}

```

C. Ejecución del Seeder

Para ejecutar el seeder, deberás modificar tu `package.json` para añadir un script de ejecución y luego usar `ts-node`.

1. Abre `package.json` y añade el script bajo "scripts"

```

"scripts": {
  "start": "ts-node src/index.ts",
  "seed": "ts-node prisma/seed.ts", <--- AÑADE ESTA LÍNEA
  "dev": "nodemon src/index.ts"
}

// ejecutar esto para agregar los super heroes
npx knex seed:run

```

2.- Ejecuta la migración y el seeder (en ese orden) en tu terminal:

1. Aplica la migración para crear las tablas
npx prisma migrate dev --name init

2. Ejecuta el script seeder
npm run seed

Una vez que el seeder se ejecute con éxito, tu base de datos tendrá **12 superhéroes** listos para ser mostrados en tu frontend en la configuración de grid 4x3.

Servicios

--Agregar un Archivo .Ts en modulo de services: “notify.service.ts”

```
// src/app/services/notify.service.ts
import { Injectable, signal } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class NotifyService {
  message = signal<string | null>(null);
  type = signal<'success' | 'error' | 'info'>('info');

  show(msg: string, type: 'success' | 'error' | 'info' = 'info') {
    this.message.set(msg);
    this.type.set(type);

    // Se limpia automáticamente tras 3.5 segundos
    setTimeout(() => {
      this.message.set(null);
    }, 3500);
  }
}
```

El Servicio de Favoritos (favorites.service.ts)

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class FavoritesService {
  private http = inject(HttpClient);
  private apiUrl = 'http://localhost:3000/api/favoritos'; // Ajusta a tu URL

  // Obtener todos los favoritos del usuario actual
  getFavorites(): Observable<any[]> {
    return this.http.get<any[]>(`${this.apiUrl}`);
  }

  // Agregar a favoritos
  addFavorite(heroId: number): Observable<any> {
```

```
    return this.http.post(`${this.apiUrl}`, { heroId });  
  }  
  
  // Eliminar de favoritos  
  removeFavorite(heroId: number): Observable<any> {  
    return this.http.delete(`${this.apiUrl}/${heroId}`);  
  }  
}
```

Agregaremos los archivos para manejo de la pestaña “Favoritos”
en el CMD ejecutar lo siguiente:
ng generate component pages/favorites/favorites --skip-tests --flat

Agregamos los 3 documentos(.ts , .html y .scss) para poder agregar un superheroe:
en el CMD en la ruta del proyecto:

ng generate component pages/add-hero/add-hero --skip-tests --flat